Functions

Advanced Programming

Brent van Bladel

Functions

- Break the program into smaller and simpler pieces.
  Increase structure, readability, and comprehension.
- Provides abstraction functionality.
  Self-documenting code.
- Enables reusability and reduces redundancy.

Function Terminology

Type: refers to type of the return value.

Signature: function name + types of the parameters list.

Prototype: type + signature; the function declaration.

Declaration vs Definition

Declaration: Prototype + optional function specifiers:

- inline
- constexpr
- consteval
- noexcept
- nodiscard
- virtual, override, final, const... (see chapter on classes)

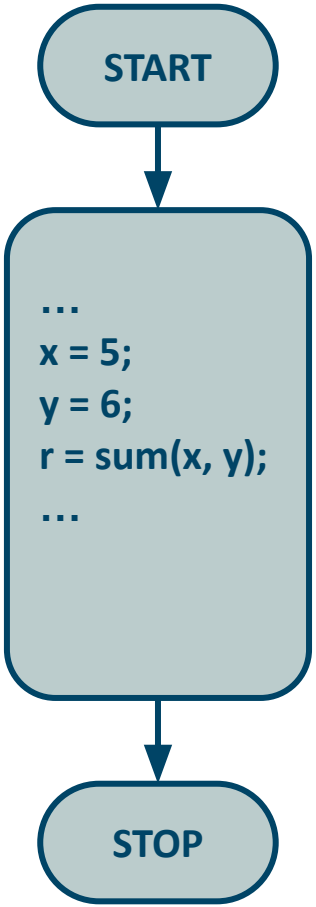
Definition: declaration + function body.

Inline Functions

```
1.  inline int sum(int a, int b) {
2.      return a + b;
3.  }
```

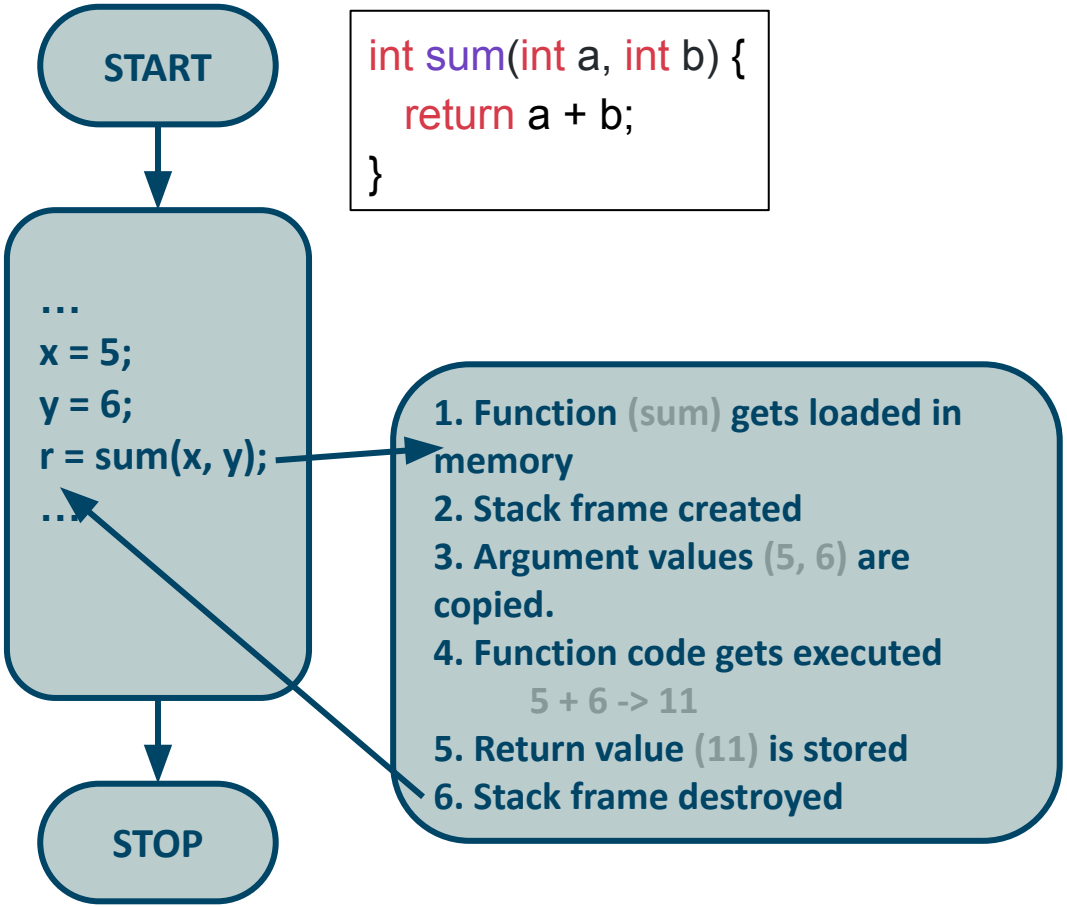
Suggestion to the compiler to expand the function in line when it is called to reduce the function call overhead.

Standard Function vs Inline Function




```
int sum(int a, int b) {
   return a + b;
}
```

Standard Function vs Inline Function

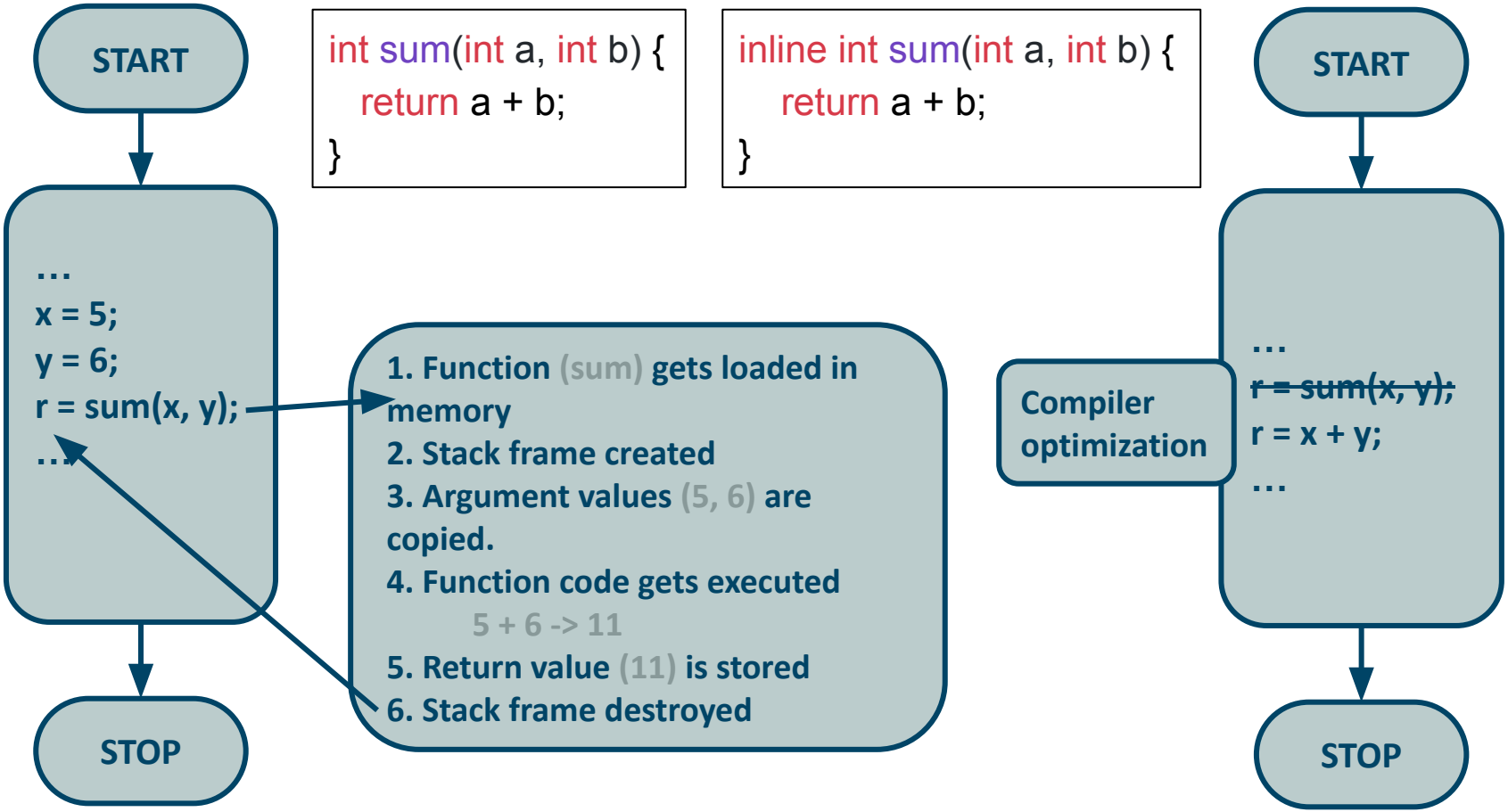



```
int sum(int a, int b) {
   return a + b;
}
```

Standard Function vs Inline Function

```
int sum(int a, int b) {
    return a + b;
}
```

```
inline int sum(int a, int b) {
    return a + b;
}
```



Inline Functions

```
1.  inline int sum(int a, int b) {
2.      return a + b;
3.  }
```

Suggestion to the compiler to expand the function in line when it is called to reduce the function call overhead.

⟶ Only a suggestion: the compiler will not inline a function if it is too big or too complex, or if it would cause side effects (example: pass by value).

Inline Functions

Advantages

- It speeds up your program by avoiding function calling overhead.
  - It saves overhead of stackframe push/pop.
  - It saves overhead of return call from a function.
- By marking it as inline, you can put a function definition in a header file.

Inline Functions

Advantages

- It speeds up your program by avoiding function calling overhead.
  - It saves overhead of stackframe push/pop.
  - It saves overhead of return call from a function.
- By marking it as inline, you can put a function definition in a header file.

Disadvantages

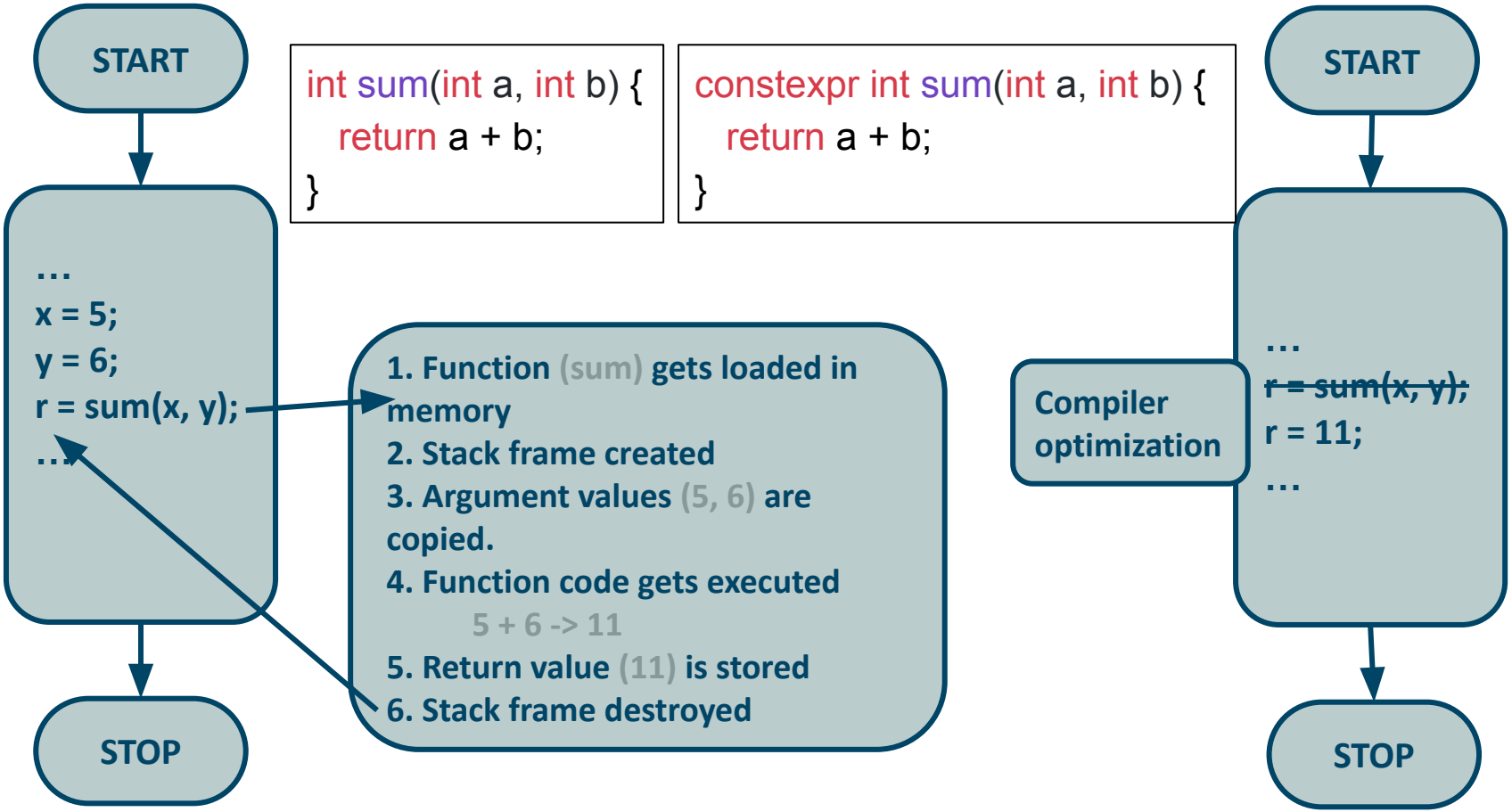
- It can increase the executable size due to code expansion.
- C++ inlining is resolved at compile time, increasing compilation time.

Constexpr Functions

```
1.  constexpr int sum(int a, int b) {
2.      return a + b;
3.  }
```

Specify that the value of a function can be evaluated at compile-time.

Standard Function vs Constexpr Function



Constexpr Functions

Advantages

- It speeds up your program by avoiding function calling overhead.
  - It saves overhead of variables push/pop on the stack.
  - It saves overhead of return call from a function.
  - It saves overhead of the calculation.
- It reduces the executable size.

Constexpr Functions

Advantages

- It speeds up your program by avoiding function calling overhead.
  - It saves overhead of variables push/pop on the stack.
  - It saves overhead of return call from a function.
  - It saves overhead of the calculation.
- It reduces the executable size.

Disadvantages

- Constexpr functions are calculated at compile time, increasing compilation time.
- Limited use. (Note that a constexpr function will still be evaluated at runtime if the arguments cannot be statically determined.)

Consteval Functions

```
1.  consteval int sum(int a, int b) {
2.      return a + b;
3.  }
```

Specify that the value of a function **must be** evaluated at compile-time, and to generate a compiler error if not possible.

Noexcept Functions

```
1.  int sum(int a, int b) noexcept {
2.      return a + b;
3.  }
```

Indicate that a function should not throw exceptions.

- Compiler can use this information to optimize.
  Example: containers such as std::vector will move their elements if the elements' move constructor is noexcept, and copy otherwise
- Other programmers can use this information to optimize.
  Example: no need to write exception handling code.

Nodiscard Functions

```
1.  [[nodiscard]] int sum(int a, int b) {
2.      return a + b;
3.  }
```

Indicate that the return value of a function should be used.

Useful for:

- A function with no effects beyond returning a certain result.
  If the result is not used, the call is useless.
- A function where the return value must be checked.
  Example: for a C-like interface that returns error codes instead of throwing.

Nodiscard Functions

```
1.  [[nodiscard]] int sum(int a, int b) {
2.      return a + b;
3.  }
4.
5.  int main() {
6.      sum(5,6);
7.  }
```

Compiler warning!

```
1.  [[nodiscard]] int sum(int a, int b) {
2.      return a + b;
3.  }
4.
5.  int main() {
6.      int x = sum(5,6);
7.      std::cout << x;
8.  }
```

No compiler warning.

Function Overloading

Functions with the same name but different arguments.

```
1.  void print(long a) { std::cout << "long: " << a; }
2.
3.  void print(double a) { std::cout << "double: " << a; }
4.
5.  int main(){
6.      print(1L);  // long: 1
7.      print(1.0); // double: 1
8.      print(1);   // error: call is ambiguous
9.  }
```

The compiler tries to find the best match.

Increased chance that an unsuitable argument will be rejected!

Function Overload Resolution

The compiler tries to find the best viable match.

Conversion Sequences

1. Exact Match a match using none or trivial conversions
2. Match via promotions short to int, float to double
3. Match via standard conversions int to double, double to int
4. Match based on user-defined conversions

Namespaces

Advanced Programming

Brent van Bladel

Introduction

Name conflicts: The use of the same name for different entities results in multiple definitions. Resulting in compilation errors.

Introduction

Name conflicts: The use of the same name for different entities results in multiple definitions. Resulting in compilation errors.

→ renaming not always possible (external libraries)

or feasible (naming conventions)

Introduction

Name conflicts: The use of the same name for different entities results in multiple definitions. Resulting in compilation errors.

→ renaming not always possible (external libraries)

or feasible (naming conventions)

Solution: **Namespaces**

Namespaces

Group logically associated entities in separate scopes.

This effectively shields one definition from another, avoiding name conflicts.

```
1.  using namespace std;
2.
3.  namespace doubleFuncs { void print(double a) { cout << "double: " << a; } }
4.
5.  namespace longFuncs {     void print(long a)     { cout << "long: " << a; }; }
6.
7.  int main() {
8.    longFuncs::print(1L);  // long: 1
9.    doubleFuncs::print(1.0); // double: 1
10.   doubleFuncs::print(1);   // double: 1
11. }
```

Accessing Namespaces

Explicit Qualification namespace-name::member-name

```
1.  using namespace std;
2.
3.  namespace doubleFuncs { void print(double a) { cout << "double: " << a; } }
4.
5.  namespace longFuncs {    void print(long a)    { cout << "long: " << a; }; }
6.
7.  int main() {
8.    longFuncs::print(1L);  // long: 1
9.    doubleFuncs::print(1.0); // double: 1
10.   doubleFuncs::print(1);   // double: 1
11. }
```

Accessing Namespaces

Using-Directives introducing a namespace in the scope

```
1.  using namespace std;
2.
3.  namespace doubleFuncs { void print(double a) { cout << "double: " << a; } }
4.
5.  namespace longFuncs {     void print(long a)     { cout << "long: " << a; }; }
6.
7.  int main() {
8.    longFuncs::print(1L);  // long: 1
9.    doubleFuncs::print(1.0); // double: 1
10.   doubleFuncs::print(1);   // double: 1
11. }
```

Accessing Namespaces

Using-Declarations introducing a synonym in the scope.

```
1.  using std::cout;
2.
3.  namespace doubleFuncs { void print(double a) { cout << "double: " << a; } }
4.
5.  namespace longFuncs {     void print(long a)     { cout << "long: " << a; }; }
6.
7.  int main() {
8.    longFuncs::print(1L);  // long: 1
9.    doubleFuncs::print(1.0); // double: 1
10.   doubleFuncs::print(1);   // double: 1
11. }
```

Accessing Namespaces

Argument Dependent Lookup (aka. Koenig lookup)

There is no **operator<<** in global namespace, but ADL examines std namespace because the left argument is in std and finds **std::operator<<(std::ostream&, const char*)**

```
using std::cout;

namespace doubleFuncs { void print(double a) { cout << "double: " << a; } }

namespace longFuncs {     void print(long a)     { cout << "long: " << a; }; }

int main() {
  longFuncs::print(1L);  // long: 1
  doubleFuncs::print(1.0); // double: 1
  doubleFuncs::print(1);   // double: 1
}
```

Unnamed Namespaces

Avoid name clashes with locally used entities.

An unnamed namespace is automatically included in the current scope.

```
1.  using namespace std;
2.
3.  namespace {
4.      void print(int a) { cout << "int: " << a; }
5.  }
6.
7.  int main() {
8.    print(1);   // int: 1
9.  }
```

Namespace Management

Nesting : create hierarchical topology.

Composition: merging together disjunct namespaces.

Selection: selecting entities from various namespaces.

Extension: supplement namespace with additional entities.

Alias: enter alternative names for a namespace.